

INFORME ANÁLISIS DE FACTIBILIDAD DE SOLUCIÓN IoT

Sistema Antirrobo Inteligente con Arduino UNO R4 WiFi

NOMBRE: [Nombre y apellido del/los integrante(s)]

CARRERA: Ingeniería en Informática / Analista Programador

ASIGNATURA: Aplicaciones Móviles para IoT (TI3V42) – Sección V-FB50-N4-P14-C1

PROFESOR: [Nombre del docente]

SEDE: Valdivia

FECHA: Octubre de 2026



Contenido

I. Introducción.....3
II. Objetivos.....4
Objetivo general.....4
Objetivos específicos.....4
III. Uso de Microcontrolador, Integrando Sensores y Actuadores.....5
3.1 Selección del Microcontrolador.....5
3.2 Integración de Sensores.....6
3.3 Integración de Actuadores.....8
3.4 Esquema de Conexión.....8
3.5 Pruebas y Validación.....9
IV. Empleo de Antenas Inalámbricas.....10
4.1 Selección de Antenas.....10
4.2 Integración con el Microcontrolador.....11
4.3 Configuración y Programación.....11
4.4 Consideraciones de Seguridad.....12
V. Aplicación Móvil e IoT.....13
5.1 Requisitos de la Aplicación.....13
5.2 Diseño de la aplicación.....13
5.3 Integración de Hardware.....14
5.4 Pruebas y validación.....15
5.5 Escalabilidad.....16
VI. Análisis de entornos posibles.....17
6.1 Identificación de entornos.....17
6.2 Evaluación de factibilidad.....17
6.3 Cumplimiento de Estándares OT.....18
6.4 Impacto de Limitaciones y Restricciones.....19
6.5 Pruebas y Validación.....19
6.6 Recomendaciones.....20
VII. Conclusiones.....20
VIII. Referencias bibliográficas.....21



I. Introducción

El presente informe analizó la factibilidad de implementar un **Sistema Antirrobo Inteligente basado en Arduino e Internet de las Cosas (IoT)**, desarrollado como proyecto integrador de la asignatura. La solución detectó intrusiones en un espacio físico combinando tres variables ambientales —sonido, oscuridad y proximidad— medidas por sensores conectados a una placa **Arduino UNO R4 WiFi**, y notificó cada evento en tiempo real a un aplicativo web con inicio de sesión, alertas por SMS, notificaciones push, cámara en vivo y reconocimiento facial.

En primer lugar, se documentó el uso del microcontrolador: los criterios técnicos con que se seleccionó la placa frente a otras alternativas, la integración de los sensores HW-485 (sonido), HW-486 (luz) y HC-SR04 (distancia), los actuadores y salidas del sistema, el esquema de conexión y las pruebas realizadas. Luego se analizó el empleo de la antena inalámbrica WiFi integrada (módulo ESP32-S3), su configuración y programación, y las medidas de seguridad aplicadas a la comunicación.

A continuación, se describió la aplicación móvil/web e IoT: sus requisitos funcionales y no funcionales, el diseño de sus interfaces, los mecanismos de comunicación con el hardware (HTTPS, Socket.IO, WebRTC, Web Push y SMS), las pruebas de validación y las funcionalidades propuestas para escalar la solución a la industria.

Finalmente, se evaluaron tres entornos posibles de implementación —vivienda, pequeño comercio y bodega o recinto industrial—, analizando su factibilidad técnica y económica, el alineamiento con estándares de Tecnología Operacional (OT), las limitaciones y restricciones de cada entorno, los métodos de prueba y validación y un conjunto de recomendaciones. El informe cierra con las conclusiones del trabajo y las referencias bibliográficas consultadas.



II. Objetivos

Objetivo general

Evaluar la factibilidad técnica y económica de implementar un sistema antirrobo IoT basado en Arduino UNO R4 WiFi, que detecta intrusiones mediante sensores y notifica en tiempo real a una aplicación web/móvil, considerando su integración de hardware, comunicación inalámbrica, seguridad y su despliegue en distintos entornos.

Objetivos específicos

- Seleccionar y justificar el microcontrolador del proyecto a partir de los requisitos técnicos de la aplicación, comparándolo con alternativas disponibles en el mercado.
- Integrar sensores y actuadores con el microcontrolador, documentando su conexión, configuración y el esquema eléctrico del prototipo.
- Configurar y programar la comunicación inalámbrica WiFi del sistema, incorporando medidas de seguridad para proteger los datos transmitidos.
- Describir los requisitos, el diseño y la integración de la aplicación móvil/web con el hardware, junto con las pruebas que validan su funcionamiento.
- Analizar al menos tres entornos de implementación, evaluando su factibilidad, el cumplimiento de estándares OT y el impacto de sus limitaciones y restricciones.
- Proponer recomendaciones de optimización y funcionalidades de escalabilidad que permitan llevar la solución a un contexto industrial.



III. Uso de Microcontrolador, Integrando Sensores y Actuadores

La Figura 1 resume la arquitectura completa de la solución. Los sensores entregan señales analógicas y digitales al Arduino UNO R4 WiFi, que ejecuta la lógica de detección localmente y envía un paquete JSON por segundo al servidor en la nube (Node.js + Express desplegado en Render). El servidor reenvía los datos en tiempo real al aplicativo mediante Socket.IO, persiste usuarios y configuraciones en MongoDB y despacha las alertas por SMS (Twilio) y notificaciones push (Web Push/VAPID).

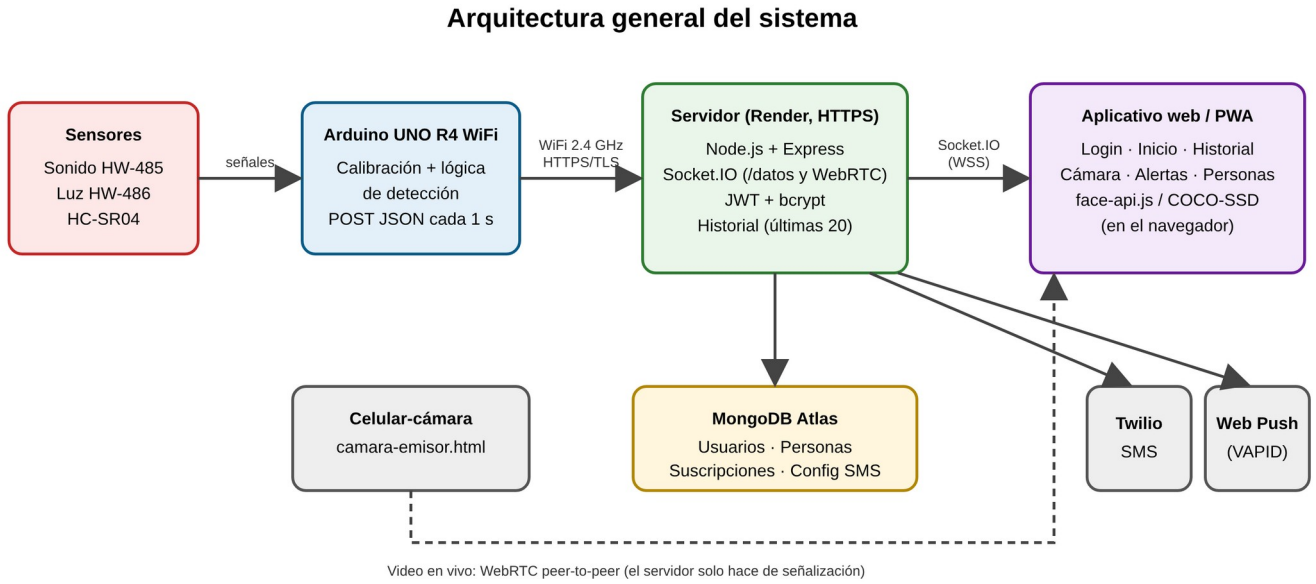


Figura 1. *Arquitectura general del sistema antirrobo IoT. Fuente: elaboración propia.*

3.1 Selección del Microcontrolador

Antes de elegir la placa se definieron los requisitos técnicos que impone la aplicación:

- **Conectividad a Internet integrada** (WiFi) para enviar datos a un servidor en la nube sin depender de un computador intermedio.
- **Soporte de TLS/HTTPS**, ya que el servidor (Render) solo acepta conexiones cifradas en el puerto 443.
- **Al menos dos entradas analógicas** (sonido y luz) y **dos pines digitales** para el sensor ultrasónico (Trig/Echo).
- **Lógica de 5 V**, para conectar directamente los módulos HW-485, HW-486 y HC-SR04 (este último entrega un pulso Echo de 5 V) sin conversores de nivel.
- **Memoria suficiente** para armar en RAM la petición HTTP completa (~450 bytes) y mantener la calibración y el estado de la alarma.
- **Ecosistema maduro** (Arduino IDE, librerías y documentación) y costo accesible para un prototipo académico.

Con estos criterios se compararon cinco alternativas (Tabla 1).

Tabla 1. *Comparación de microcontroladores evaluados (precios referenciales en tiendas de electrónica chilenas, 2026).*

Parámetro	Arduino UNO R4 WiFi (elegido)	ESP32 DevKit V1	Raspberry Pi Pico W	Arduino UNO R3 + ESP-01	Raspberry Pi 4
CPU	Renesas RA4M1, Cortex-M4 48 MHz + ESP32-S3 (radio)	Xtensa LX6 doble núcleo 240 MHz	RP2040 doble núcleo 133 MHz	ATmega328P 16 MHz + ESP8266	Cortex-A72 4 núcleos 1,5 GHz



Parámetro	Arduino UNO R4 WiFi (elegido)	ESP32 DevKit V1	Raspberry Pi Pico W	Arduino UNO R3 + ESP-01	Raspberry Pi 4
RAM / Flash	32 KB / 256 KB	520 KB / 4 MB	264 KB / 2 MB	2 KB / 32 KB	2–8 GB / microSD
WiFi / BLE	Sí / Sí (ESP32-S3)	Sí / Sí	Sí / Sí	Sí (por AT) / No	Sí / Sí
TLS (HTTPS)	Sí, WiFiSSLClient	Sí	Sí	Muy limitado (2 KB RAM)	Sí
Nivel lógico	5 V	3,3 V	3,3 V	5 V	3,3 V
ADC	6 canales, hasta 14 bits	18 canales 12 bits (no lineal)	3 canales 12 bits	6 canales 10 bits	No tiene
Compatibilidad con HC-SR04 (Echo 5 V)	Directa	Requiere divisor	Requiere divisor	Directa	Requiere divisor
Precio ref. (CLP)	≈ $32.000	≈ $7.000	≈ $9.000	≈ $18.000	≈ $65.000
Evaluación	Cumple todos los requisitos	Cumple, pero exige adaptar niveles	Cumple, pero exige adaptar niveles	No cumple TLS ni memoria	Sobredimensionado y sin ADC

Se seleccionó el **Arduino UNO R4 WiFi** porque es la única alternativa que reúne, en una sola placa y sin componentes adicionales, lógica de 5 V compatible con todos los sensores, conversor analógico-digital de buena resolución y un coprocesador de radio **ESP32-S3** capaz de establecer conexiones TLS hacia el servidor en la nube. Aunque el ESP32 DevKit tiene más potencia y menor precio, su lógica de 3,3 V obligaba a agregar divisores de tensión para el HC-SR04 y su ADC es poco lineal, lo que complica la calibración relativa de los sensores. La Raspberry Pi 4 se descartó por estar sobredimensionada (sistema operativo completo, mayor consumo y costo) y por no contar con entradas analógicas, y el UNO R3 con ESP-01 por no disponer de memoria suficiente para manejar TLS.

Una característica relevante de la placa elegida es que el WiFi reside en un chip separado (ESP32-S3) con el que el RA4M1 se comunica por UART. Cada operación de red es un viaje de ida y vuelta a ese chip, por lo que el firmware se diseñó para minimizar esas llamadas, como se detalla en la sección 4.3.

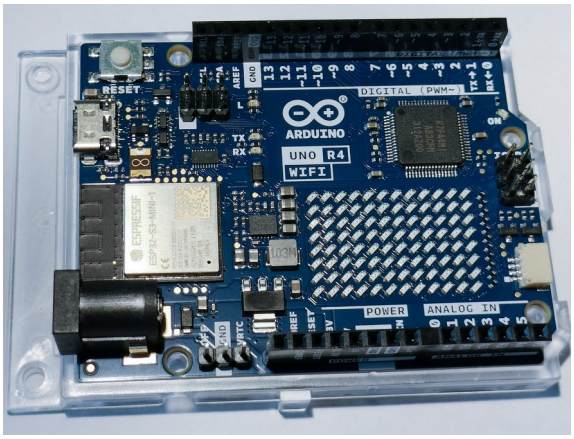


Figura 2. *Placa Arduino UNO R4 WiFi utilizada. Fuente: Lomrjyo / Wikimedia Commons (CC BY-SA 4.0).*

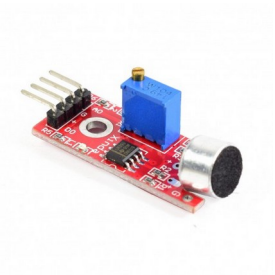
3.2 Integración de Sensores

El sistema utilizó tres sensores complementarios. Ninguno basta por sí solo para confirmar una intrusión, pero la coincidencia de los tres en el mismo instante —**ruido fuerte, oscuridad y un objeto a menos de 50 cm**— reduce de forma importante las falsas alarmas (Tabla 2).



Tabla 2. Sensores integrados al microcontrolador.

Sensor	Principio	Pines	Señal / medición	Propósito
HW-485 (sonido)	Micrófono electret + comparador LM393 con potenciómetro	AO → A0 DO → D2 VCC 5 V, GND	Amplitud pico a pico en ventana de 40 ms (0–1023)	Detectar golpes, rotura de vidrios o forzamiento
HW-486 (luz)	Fotorresistencia (LDR) en divisor de tensión	AO → A1 VCC 5 V, GND	Lectura ADC: valor más alto = más oscuro	Detectar ambiente a oscuras (intrusión nocturna o luces apagadas)
HC-SR04 (distancia)	Ultrasonido 40 kHz, tiempo de vuelo del eco	Trig ← D9 Echo → D10 VCC 5 V, GND	Distancia en cm (2–400 cm); timeout de 12 ms ≈ 2 m	Detectar presencia a menos de 50 cm del punto vigilado



HW-485 (sonido)



HW-486 (luz)



HC-SR04 (ultrasónico)

Figura 3. Módulos sensores utilizados. Fuente: presentación del proyecto (HC-SR04: Suyash Dwivedi / Wikimedia, CC BY-SA 4.0).

Configuración de cada sensor

- **Sonido (HW-485):** un analogRead aislado captura un punto al azar de la onda de audio, por lo que el mismo ruido se lee alto o bajo según el instante. Por ello la función medirSonidoPicoAPico() muestrea A0 durante 40 ms y calcula la diferencia entre el máximo y el mínimo, una medida estable y comparable entre módulos. La salida digital DO se conectó a D2 y se configuró como entrada (pinMode INPUT) como respaldo; la detección usa la señal analógica.
- **Luz (HW-486):** el divisor de tensión de la LDR entrega un valor mayor cuanto menor es la luz. La condición “sin luz” se cumple cuando la lectura supera el umbral calibrado, el cual se limita a un máximo de 950 para que, si el sistema se calibra a oscuras, la alarma no quede “sorda”.
- **Distancia (HC-SR04):** D9 genera un pulso de 10 µs en Trig y pulseIn() mide en D10 la duración del eco. La distancia se obtiene como d = t × 0,0343 / 2 (cm). El timeout se fijó en 12.000 µs (≈ 2 m) para no bloquear el ciclo cuando no hay obstáculos; sin eco se informa 999 cm.

Calibración automática de umbrales

Durante las pruebas se observó que dos módulos del mismo modelo entregaban lecturas muy distintas ante el mismo estado físico (por ejemplo, 600 en silencio en uno y 300 en otro). Por esa razón **no se usaron umbrales fijos**: al encender, la función calibrar() toma 50 muestras (~2 s) y obtiene el nivel de reposo y la fluctuación de cada sensor. El umbral se calcula como:

```
umbral = reposo + max( K × fluctuación , reposo × fracción , piso )
  Sonido: K = 4,0   fracción = 1,00   piso = 15
  Luz:    K = 4,0   fracción = 0,25   piso = 60   (máximo 950)
```

El término de fluctuación compensa diferencias de ganancia entre módulos, la fracción del reposo compensa diferencias de offset y el piso evita que, en un ambiente muy silencioso, el umbral quede pegado al reposo. Además, cada 60 s y solo mientras no hay alarma, la función adaptarDeriva() ajusta suavemente el reposo (promedio móvil 15/16) para seguir los cambios de luz y ruido de fondo a lo largo del día. La distancia de 50 cm sí es fija, porque corresponde a centímetros reales medidos por tiempo de eco.



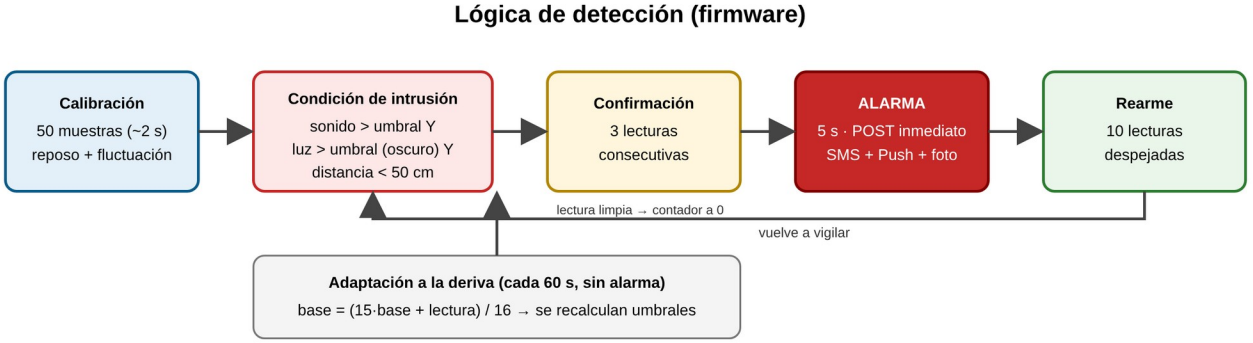


Figura 4. *Lógica de detección implementada en el firmware. Fuente: elaboración propia.*

3.3 Integración de Actuadores

En la versión final, las salidas del sistema se concentraron en actuadores remotos gestionados a través de la nube, y el microcontrolador controla directamente el pin de disparo del sensor ultrasónico. La Tabla 3 resume cada actuador o salida, su conexión con el microcontrolador y su función.

Tabla 3. *Actuadores y salidas del sistema.*

Actuador / salida	Conexión con el microcontrolador	Funcionamiento
Pin Trig del HC-SR04 (salida digital D9)	Directa, pinMode(OUTPUT)	Genera el pulso de 10 µs que inicia cada medición ultrasónica (cada 50 ms).
Alarma sonora y visual en el aplicativo	Arduino → HTTPS → servidor → Socket.IO	Al recibir alarma = true, la tarjeta de estado pasa a rojo, se muestra qué sensor se activó y se emite un pitido repetido (Web Audio API, cada 1,2 s) que el usuario puede silenciar.
Mensaje SMS	Servidor → API de Twilio	Se envía “ALERTA DE INTRUSION – revise la aplicación para más detalles” una sola vez por cada alarma nueva al número configurado.
Notificación push	Servidor → Web Push (VAPID) → service worker	Llega al teléfono aunque la aplicación esté cerrada; incluye las lecturas de sonido, luz y distancia.
Captura de fotografía	Servidor ← celular-cámara	El celular que actúa como cámara captura una foto (480 px) al activarse la alarma, que queda adjunta al historial.

La alarma tiene una duración de **5 s** (DURACION_ALARMA). Al dispararse, el firmware envía de inmediato un POST al servidor sin esperar el siguiente ciclo de 1 s; al terminar, entra en estado de **rearme** y no puede volver a dispararse hasta registrar 10 lecturas consecutivas despejadas, lo que evita notificaciones repetidas durante una misma intrusión.

En una primera versión se integró una **pantalla LCD 16×2 por I2C** como actuador local. Sin embargo, las pruebas mostraron que el bus I2C quedaba bloqueado hasta ~17 s en algunos ciclos, congelando la detección, por lo que se retiró y toda la retroalimentación se trasladó al aplicativo. Como mejora se propone incorporar un **buzzer activo o sirena de 12 V** controlada por un pin digital (por ejemplo D6) a través de un transistor NPN (2N2222) o un módulo relé con diodo de protección, para disponer de una alarma disuasiva local que funcione aun sin conexión a Internet (ver sección 6.6).

3.4 Esquema de Conexión

La Figura 5 muestra el esquema de conexión del prototipo. Todos los módulos se alimentan desde el pin de 5 V de la placa y comparten tierra común; la placa se alimenta por USB-C. La Tabla 4 detalla la asignación de pines.



Esquema de conexión – Sistema Antirrobo IoT

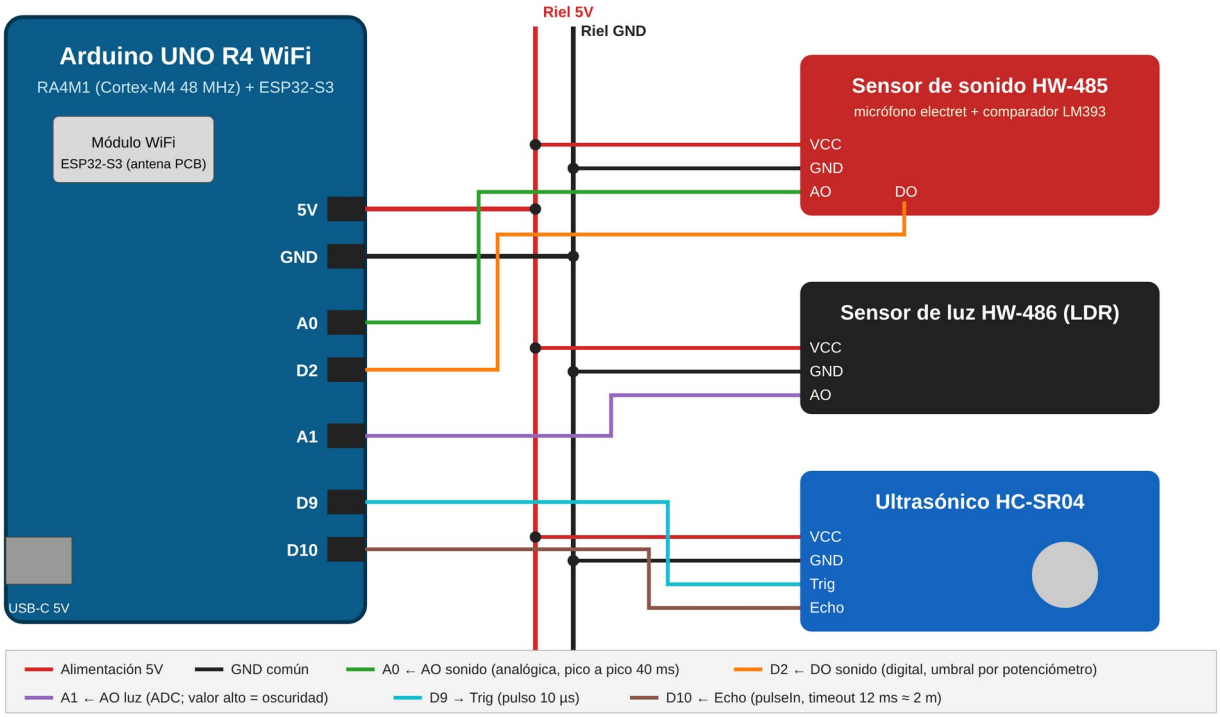


Figura 5. *Esquema de conexión del sistema antirrobo. Fuente: elaboración propia.*

Tabla 4. *Asignación de pines del prototipo.*

Componente	Pin del módulo	Pin Arduino	Tipo	Configuración en el firmware
HW-485	VCC / GND	5V / GND	Alimentación	—
HW-485	AO	A0	Entrada analógica	PIN_SONIDO; ventana pico a pico 40 ms
HW-485	DO	D2	Entrada digital	PIN_SONIDO_DO; pinMode(INPUT)
HW-486	VCC / GND	5V / GND	Alimentación	—
HW-486	AO	A1	Entrada analógica	PIN_LUZ; analogRead
HC-SR04	VCC / GND	5V / GND	Alimentación	—
HC-SR04	Trig	D9	Salida digital	PIN_TRIG; pulso 10 µs
HC-SR04	Echo	D10	Entrada digital	PIN_ECHO; pulseIn timeout 12 ms
Arduino	USB-C	—	Alimentación 5 V	Serial a 115.200 baudios para diagnóstico

3.5 Pruebas y Validación

Las pruebas de hardware se realizaron con el monitor serie del Arduino IDE (115.200 baudios). Con la opción DEBUG_TIMING activada, el firmware imprime en cada vuelta una línea con el tiempo de cada etapa (WiFi, sensores, distancia y envío), las lecturas frente a sus umbrales, el contador de confirmaciones y el estado de la red, por ejemplo:

```
[t] wifi=1 sens=40 dist=3 env=6 | S:12/27 L:410/512 D:999cm | C:0/3 | WiFi OK
[t] wifi=1 sens=40 dist=2 env=5 | S:88/27[!] L:640/512[!] D:32cm[!] | C:3/3 | WiFi OK | ALARMA
```



Tabla 5. Pruebas de integración de sensores y actuadores.

N°	Prueba	Procedimiento	Criterio de aceptación	Resultado
P1	Calibración	Encender en silencio y luz normal; revisar reposo y umbrales impresos	Umbrales calculados sobre el reposo de cada módulo	Cumple
P2	Sensor de sonido	Aplaudir y golpear la mesa cerca del micrófono	Lectura pico a pico supera el umbral ([!])	Cumple
P3	Sensor de luz	Tapar la LDR y apagar la luz de la sala	Lectura supera el umbral de “sin luz”	Cumple
P4	Sensor ultrasónico	Acercar la mano/objeto a 100, 50 y 30 cm y comparar con huincha	Error < 2 cm; alerta solo bajo 50 cm	Cumple
P5	Condición combinada	Activar solo 1 o 2 condiciones a la vez	No se dispara la alarma	Cumple
P6	Confirmación	Generar las 3 condiciones durante 1 lectura aislada y luego de forma sostenida	Alarma solo con 3 lecturas consecutivas	Cumple
P7	Duración y rearme	Mantener la intrusión después de la alarma	Alarma de 5 s; no re-dispara hasta 10 lecturas despejadas	Cumple
P8	Robustez de módulos	Reemplazar el sensor de sonido por otro módulo HW-485	Funciona sin reprogramar gracias a la calibración	Cumple
P9	Tiempo de ciclo	Observar tiempos [t] durante 10 minutos	Ciclo ≈ 50 ms sin bloqueos largos	Cumple tras retirar la LCD

Las pruebas permitieron detectar y corregir problemas relevantes: el bloqueo de ~17 s producido por la LCD I2C (se retiró), las lecturas dispares entre módulos (se implementó la calibración automática) y las falsas alarmas por picos aislados (se exigieron 3 lecturas consecutivas). Con estas correcciones se validó que la integración de sensores y actuadores funciona de manera estable y repetible.

IV. Empleo de Antenas Inalámbricas

4.1 Selección de Antenas

Para la comunicación inalámbrica se evaluaron las tecnologías más comunes en IoT, considerando alcance, velocidad, consumo, interferencias, compatibilidad con el microcontrolador y necesidad de infraestructura adicional (Tabla 6).

Tabla 6. Comparación de tecnologías inalámbricas.

Criterio	WiFi 2,4 GHz (elegido)	Bluetooth LE	LoRa / LoRaWAN	Celular 4G/LTE-M	Zigbee
Alcance típico	30–50 m interior	10–30 m	2–15 km	Cobertura del operador	10–100 m (malla)
Velocidad	Hasta 150 Mbit/s (802.11n)	1–2 Mbit/s	0,3–50 kbit/s	~1 Mbit/s (LTE-M)	250 kbit/s
Consumo	Medio-alto	Muy bajo	Muy bajo	Medio	Bajo
Acceso a Internet	Directo vía router	Requiere teléfono/gateway	Requiere gateway	Directo (SIM)	Requiere coordinador
Interferencias	Banda compartida, congestionable	Banda 2,4 GHz	Baja (sub-GHz)	Baja	Banda 2,4 GHz
Compatibilidad UNO R4 WiFi	Integrado (ESP32-S3)	Integrado (ESP32-S3)	Módulo externo	Módulo externo	Módulo externo
Costo adicional	$0	$0	Módulo + gateway	Módulo + plan mensual	Módulos + coordinador

Se seleccionó **WiFi IEEE 802.11 b/g/n en 2,4 GHz**, utilizando la **antena PCB integrada del módulo ESP32-S3** de la placa. Las razones fueron: (1) el sistema necesita enviar datos a un servidor en Internet y el WiFi lo permite directamente a través del router existente, sin gateways; (2) el ancho de banda es holgado para un



paquete JSON de ~200 bytes por segundo; (3) viene integrado en la placa, sin costo adicional ni cableado; y (4) la mayoría de los entornos objetivo (hogares, comercios, bodegas) ya cuentan con cobertura WiFi. BLE se descartó porque exige un teléfono cerca que actúe de puente; LoRa porque requiere un gateway y su baja tasa no soporta el envío cada segundo; y 4G por el costo recurrente del plan de datos, aunque se recomienda como respaldo en entornos industriales.

La antena PCB es omnidireccional en el plano de la placa, con una ganancia aproximada de 2 dBi, suficiente para enlaces interiores de algunas decenas de metros. Su principal limitación es la atenuación por muros de hormigón y estructuras metálicas, por lo que en recintos grandes se recomienda un repetidor o un módulo con conector para antena externa (sección 6.4).

4.2 Integración con el Microcontrolador

En el UNO R4 WiFi la antena y la radio forman parte del módulo ESP32-S3 soldado en la misma placa, que se comunica con el microcontrolador principal RA4M1 por UART. El procedimiento de integración fue el siguiente:

1. Instalar en Arduino IDE el paquete de placas **“Arduino UNO R4 Boards”**, que incluye la librería **WiFiS3**.
2. Actualizar el firmware del ESP32-S3 con la herramienta de actualización de firmware del IDE, para asegurar compatibilidad con TLS y los certificados raíz actualizados.
3. Incluir la librería con #include <WiFiS3.h> y verificar al arrancar que el módulo responda: si WiFi.status() devuelve WL_NO_MODULE se informa “revisar firmware del ESP32-S3”.
4. Configurar la red (SSID y contraseña) de un punto de acceso de 2,4 GHz con cifrado WPA2. En las pruebas se usó un hotspot de teléfono móvil.
5. Ubicar la placa de modo que la antena (esquina del módulo ESP32-S3) no quede tapada por metal ni junto a fuentes de interferencia, y verificar la intensidad de señal con WiFi.RSSI() (se recomienda mejor que −70 dBm).
6. Crear un cliente seguro WiFiSSLClient para conectarse al servidor por HTTPS (puerto 443).

Se eligió esta forma de integración porque no requiere hardware adicional, utiliza las librerías oficiales del fabricante y el cliente WiFiSSLClient realiza el cifrado TLS dentro del ESP32-S3, liberando al RA4M1 de esa carga criptográfica.

4.3 Configuración y Programación

La programación de la comunicación inalámbrica se organizó en cuatro funciones principales del sketch antirobo.ino:

- **conectarWiFi():** se ejecuta en setup(), llama a WiFi.begin(SSID, PASS) y espera hasta 30 intentos de 500 ms; informa la IP local o continúa sin red para no detener la detección.
- **reintentarWiFi():** si la conexión se pierde, reintenta con **backoff exponencial** (5 s, 10 s, 20 s… hasta 60 s). Como WiFi.begin() es bloqueante, espaciar los reintentos evita que el ciclo de detección quede congelado cuando el punto de acceso no está disponible.
- **enviarDatos():** cada 1 s (INTERVALO_ENVIO) y de inmediato al activarse o terminar una alarma, arma en un buffer de memoria la petición HTTP completa y la envía con **un único write()**. Con print() el núcleo enviaba un byte por llamada (~290 viajes al ESP32-S3 por envío); con el buffer es solo uno. La conexión TLS se reutiliza con **Connection: keep-alive** para no repetir el handshake en cada envío; si el servidor no responde, se aplica backoff de 1 a 30 s.



- **verificarRespuestaPendiente():** la respuesta del servidor no se espera; se lee al inicio del envío siguiente, consumiendo solo lo que ya llegó, y se valida el código HTTP (2xx). Así el ciclo nunca se bloquea esperando a la red.

El estado WiFi.status() se consulta una sola vez por vuelta y se reutiliza. La petición enviada tiene la siguiente forma:

```
POST /api/datos HTTP/1.1
Host: antirobo-server.onrender.com
Content-Type: application/json
Content-Length: 150
Connection: keep-alive

{"sonido":12,"sonidoUmbral":27,"luz":410,"luzUmbral":512,"distancia":999,
 "alarma":false,"confirmadas":0,"objetivo":3,"alertas":0,"segundos":3600}
```

El JSON incluye las lecturas, los umbrales calibrados (para que el aplicativo los muestre sin conocerlos de antemano), el estado de la alarma, el contador de confirmaciones, el total de alertas y el tiempo de funcionamiento, que se lleva con un contador propio para no reiniciarse al desbordar millis() a los ~49,7 días.

4.4 Consideraciones de Seguridad

La Tabla 7 resume las medidas de seguridad implementadas para proteger la comunicación inalámbrica y el acceso al sistema.

Tabla 7. *Medidas de seguridad implementadas.*

Medida implementada	Propósito / cómo protege al sistema
Cifrado WPA2 en la red WiFi	Impide que terceros se asocien a la red o lean el tráfico a nivel de enlace.
HTTPS/TLS extremo a extremo (puerto 443) con WiFiSSLClient	Cifra los datos del Arduino hacia el servidor y valida el certificado del servidor, evitando lectura o alteración en tránsito.
Inicio de sesión obligatorio	El aplicativo, la página de la cámara y todas las APIs de consulta redirigen al login (o responden 401) sin sesión válida. No existe registro público de usuarios.
Contraseñas con bcrypt	Se almacena solo el hash con sal (10 rondas), nunca la contraseña real.
Sesión con JWT en cookie httpOnly + sameSite	Token firmado con 7 días de validez; httpOnly impide que un script lo robe (XSS) y sameSite mitiga CSRF; secure en producción.
Socket.IO autenticado	Ambos canales en tiempo real (datos y señalización de video) verifican el JWT antes de aceptar la conexión.
Secretos en variables de entorno	JWT_SECRET, claves VAPID, credenciales de Twilio y URI de MongoDB quedan fuera del código fuente.
Validación de entradas	Teléfono con formato E.164, descriptor facial de exactamente 128 valores, fotos solo con prefijo data:image/.
Video WebRTC punto a punto	El video viaja cifrado (DTLS-SRTP) directo entre celular y navegador; el servidor solo hace señalización.
Privacidad biométrica	Se almacena la huella facial (128 números), no la fotografía de la persona registrada.

Del análisis se identificaron mejoras que se sugieren para una versión productiva:

- **Autenticar al dispositivo:** la ruta POST /api/datos no exige credenciales, por lo que alguien que conozca la URL podría enviar datos falsos. Se sugiere que cada Arduino envíe una clave de API en un encabezado (por ejemplo X-Device-Key) o una firma HMAC-SHA256 del cuerpo con marca de tiempo, validada por el servidor.
- **Sacar las credenciales WiFi del código:** el SSID y la contraseña están escritos en el sketch. Se recomienda moverlos a un archivo arduino_secrets.h excluido del repositorio o usar un modo de aprovisionamiento.



- **Fijar el certificado (certificate pinning)** del servidor y aplicar limitación de intentos (rate limiting) en el login y cabeceras de seguridad (helmet).
- **Segmentar la red:** conectar los dispositivos IoT a una red o VLAN separada de los equipos de usuarios, con WPA3 cuando el hardware lo soporte.

V. Aplicación Móvil e IoT

5.1 Requisitos de la Aplicación

La aplicación se desarrolló como una aplicación web progresiva (PWA) adaptada a pantallas de teléfono, accesible desde cualquier navegador con Internet. Sus requisitos se presentan en las Tablas 8 y 9.

Tabla 8. *Requisitos funcionales.*

ID	Requisito funcional
RF-01	Permitir el acceso solo a usuarios autenticados mediante usuario y contraseña, con cierre de sesión.
RF-02	Mostrar en tiempo real el estado del sistema (Vigilando / Alarma) y las lecturas de sonido, movimiento (distancia) y luz con sus umbrales.
RF-03	Indicar qué sensores se activaron cuando hay una alarma y emitir una alarma sonora silenciable.
RF-04	Registrar un historial de las últimas 20 alertas con fecha, lecturas y fotografía.
RF-05	Enviar notificaciones push al dispositivo, incluso con la aplicación cerrada, y permitir una notificación de prueba.
RF-06	Configurar un número telefónico y enviar un SMS en cada alarma nueva, con botón de SMS de prueba.
RF-07	Ver la cámara en vivo transmitida desde un celular y capturar una foto automática al activarse la alarma.
RF-08	Reconocer rostros de personas conocidas, marcar “Desconocido” y detectar personas aunque no se vea su rostro.
RF-09	Registrar, listar y eliminar personas conocidas a partir de una captura y un nombre.

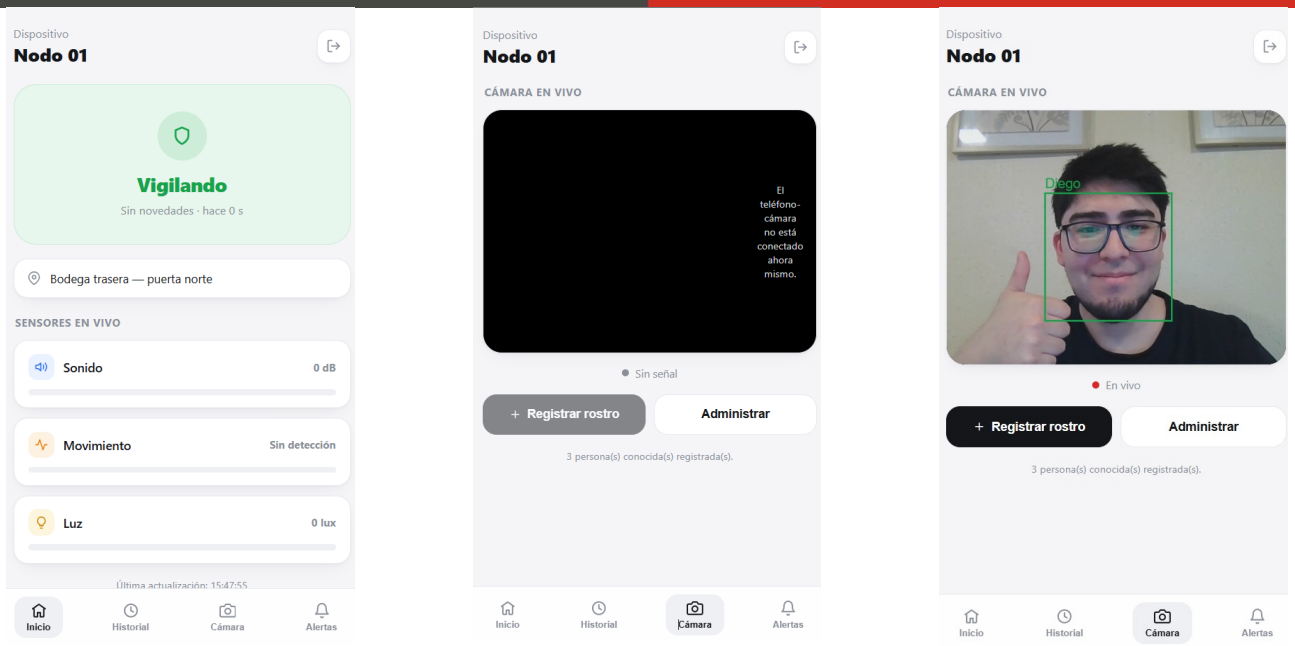
Tabla 9. *Requisitos no funcionales.*

ID	Requisito no funcional
RNF-01	Latencia: los datos del sensor deben verse en el aplicativo en ≈ 1 s (envío cada 1 s + push por Socket.IO, con respaldo cada 3 s).
RNF-02	Seguridad: comunicación HTTPS, contraseñas con bcrypt, sesión JWT en cookie httpOnly, secretos en variables de entorno.
RNF-03	Disponibilidad: accesible desde cualquier lugar con Internet (despliegue en Render); el firmware reintenta la conexión automáticamente.
RNF-04	Usabilidad: interfaz móvil con barra de navegación inferior de 4 pestañas e indicadores por color.
RNF-05	Privacidad: el reconocimiento facial se ejecuta en el navegador y solo se guardan descriptores de 128 valores.
RNF-06	Compatibilidad: navegadores modernos en Android, iOS (como app agregada a inicio) y escritorio.
RNF-07	Rendimiento: modelos de IA servidos con caché de 30 días para no descargar ~7 MB en cada visita.

5.2 Diseño de la aplicación

La interfaz se diseñó con un estilo limpio de tarjetas, pensada para usarse con una mano en el teléfono. Una barra inferior permite navegar entre **Inicio, Historial, Cámara y Alertas**, más una vista secundaria de **Personas conocidas**.





a) Inicio *b) Cámara sin señal* *c) Cámara con reconocimiento*

Figura 6. *Interfaces del aplicativo en un teléfono móvil. Fuente: capturas del sistema.*

- **Inicio de sesión:** formulario de usuario y contraseña. Sin sesión válida, cualquier página o API redirige aquí.
- **Inicio (Figura 6a):** tarjeta de estado que muestra “Vigilando” en verde (con el tiempo desde la última novedad) o “Alarma” en rojo; ubicación del nodo (“Bodega trasera — puerta norte”); y tres tarjetas de sensores en vivo (sonido, movimiento y luz) con barras de nivel. Durante una alarma aparece la sección “Qué se activó”, con el valor de cada sensor frente a su umbral y los botones **Silenciar** y **Ver cámara**.
- **Historial:** lista de las últimas alertas con fecha y hora, lecturas de cada sensor y la fotografía capturada, junto al contador de alertas totales.
- **Cámara (Figuras 6b y 6c):** muestra el video en vivo del celular-cámara. Sobre la imagen se dibujan recuadros con el nombre de la persona reconocida (o “Desconocido”) y recuadros de “persona” detectada por COCO-SSD cuando no se ve el rostro. Incluye los botones **Registrar rostro** y **Administrar**.
- **Personas conocidas:** lista de personas registradas con opción de eliminarlas.
- **Alertas:** activación de notificaciones push con botón de prueba, y configuración del número para SMS con botones **Guardar número** y **SMS de prueba**.
- **Emisor de cámara:** página que se abre en un celular dedicado, solicita la cámara trasera y transmite el video por WebRTC; además envía la foto automática cuando se activa la alarma.

5.3 Integración de Hardware

La aplicación no se comunica directamente con el Arduino, sino a través del servidor, lo que permite acceder al sistema desde cualquier red. Los métodos utilizados se describen en la Tabla 10.

Tabla 10. *Métodos de comunicación entre el aplicativo y el hardware.*

Enlace	Método / protocolo	Descripción
Arduino → Servidor	HTTP POST + JSON sobre TLS (REST)	Ruta /api/datos; un paquete por segundo y uno inmediato en cada cambio de alarma.



Enlace	Método / protocolo	Descripción
Servidor → Aplicativo	Socket.IO (WebSocket seguro), namespace /datos	El servidor emite el evento “datos” con el estado y el historial apenas recibe un POST.
Aplicativo → Servidor	HTTP GET /api/datos cada 3 s	Refresco de respaldo si el socket se desconecta.
Celular-cámara ↔ Aplicativo	WebRTC (P2P) + señalización Socket.IO + STUN	Oferta, respuesta y candidatos ICE se intercambian vía servidor; el video viaja directo.
Servidor → Teléfono	Web Push (VAPID) y SMS (Twilio)	Avisos una vez por cada alarma nueva, aun con la app cerrada.
Servidor ↔ Base de datos	MongoDB (Mongoose)	Usuarios, personas conocidas, suscripciones push y configuración del SMS.

El servidor detecta el **flanco de subida** de la alarma (alarma pasa de false a true) para registrar el evento en el historial y enviar las notificaciones una sola vez, aunque el Arduino siga enviando alarma = true mientras dura el evento.

5.4 Pruebas y validación

Tabla 11. *Pruebas realizadas a la aplicación.*

N°	Prueba	Procedimiento	Resultado esperado	Resultado
A1	Login	Ingresar con credenciales correctas e incorrectas	Acceso solo con credenciales válidas; error 401 en caso contrario	Cumple
A2	Protección de rutas	Abrir /, /api/datos y /camara-emisor.html sin sesión	Redirección al login o respuesta 401	Cumple
A3	Datos en tiempo real	Variar los sensores y observar el Inicio	Cambio visible en ≈ 1 s	Cumple
A4	Alarma	Provocar una intrusión real con los tres sensores	Tarjeta roja, detalle de sensores y pitido; botón Silenciar funcional	Cumple
A5	Historial con foto	Generar una alarma con el celular-cámara conectado	Nueva entrada con fecha, lecturas y fotografía	Cumple
A6	Notificación push	Botón “Enviar notificación de prueba” y alarma real con la app cerrada	Notificación recibida en el teléfono	Cumple (en iPhone, agregada a inicio)
A7	SMS	Guardar número y pulsar “SMS de prueba”	SMS recibido (Figura 7)	Cumple
A8	Cámara en vivo	Abrir el emisor en un celular y la pestaña Cámara	Video fluido por WebRTC	Cumple
A9	Reconocimiento facial	Registrar un rostro y presentarse frente a la cámara	Recuadro con el nombre; “Desconocido” para otras personas	Cumple (Figura 6c)
A10	Reconexión	Cortar el WiFi del Arduino y restablecerlo	El firmware reconecta solo y los datos se reanudan	Cumple

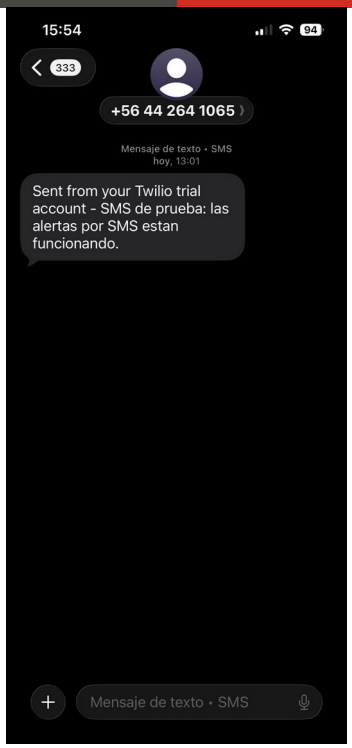


Figura 7. *Evidencia de recepción del SMS de prueba enviado por el sistema. Fuente: captura del teléfono.*

5.5 Escalabilidad

Pensando en una implementación en la industria, se proponen las siguientes funcionalidades:

- **Múltiples nodos y zonas:** identificar cada Arduino con un ID de dispositivo, agruparlos por sitio y zona, y mostrar un mapa o plano con el estado de cada punto.
- **Usuarios y roles:** administrador, guardia y supervisor, con permisos diferenciados y registro de auditoría de quién silenció o revisó cada alarma.
- **Armado y desarmado remoto y por horario:** programar turnos (por ejemplo, armar fuera del horario laboral) y un modo “mantenimiento”.
- **Historial persistente y reportes:** guardar todas las alertas en la base de datos (hoy se mantienen las 20 últimas en memoria), con exportación a CSV/PDF y estadísticas de eventos por zona y hora.
- **Protocolo MQTT y estándares industriales:** migrar el envío a MQTT sobre TLS (más liviano y con calidad de servicio) e integrarse con SCADA/BMS mediante OPC UA o Modbus TCP.
- **Integración con terceros:** envío a centrales de monitoreo, WhatsApp/Telegram, correo electrónico y webhooks.
- **Actualización remota (OTA)** del firmware y monitoreo de salud de cada nodo (RSSI, tiempo activo, último reporte, batería).
- **Más sensores y cámaras:** sensores PIR, magnéticos de puerta, de vibración y varias cámaras IP, con grabación de clips en la nube.
- **Analítica con IA:** aprendizaje de patrones normales por sitio para ajustar umbrales y reducir aún más las falsas alarmas.



VI. Análisis de entornos posibles

6.1 Identificación de entornos

Se identificaron tres entornos en los que la solución puede implementarse, cada uno con características distintas de infraestructura, riesgo y exigencias (Tabla 12).

Tabla 12. *Entornos posibles de implementación.*

Aspecto	Entorno 1: Vivienda / departamento	Entorno 2: Pequeño comercio u oficina	Entorno 3: Bodega o recinto industrial
Descripción	Protección de accesos (puerta principal, ventana) mientras los ocupantes no están o duermen.	Protección del local fuera del horario de atención (caja, bodega trasera, puerta de servicio).	Protección de galpones, pañoles de herramientas o zonas de acopio en plantas, faenas o centros logísticos.
Características	WiFi doméstico disponible; espacios pequeños; ambiente silencioso de noche.	WiFi comercial; horarios definidos; ruido de calle; vitrinas iluminadas.	Grandes superficies; estructuras metálicas; ruido de maquinaria; polvo y humedad; red OT/IT existente.
Ventajas	Bajo costo frente a alarmas monitoreadas; instalación simple; aviso inmediato al celular.	Historial con foto como evidencia; varios usuarios pueden recibir avisos; sin mensualidad de monitoreo.	Complementa la vigilancia existente; datos centralizados; escalable a muchos puntos.
Desventajas	Depende del WiFi y de la energía del hogar; mascotas pueden generar eventos.	La iluminación nocturna de vitrinas puede impedir la condición de oscuridad.	Cobertura WiFi limitada; ruido constante; requiere hardware de grado industrial y cumplimiento de normas OT.

6.2 Evaluación de factibilidad

Factibilidad técnica

Vivienda: alta. El prototipo funciona tal como fue probado: basta un enchufe USB y la red WiFi del hogar. La lógica de tres condiciones se ajusta bien a intrusiones nocturnas en un espacio silencioso y oscuro.

Pequeño comercio: alta a media. Es viable técnicamente, pero la luz de vitrinas y el ruido de la calle exigen revisar la ubicación de los sensores (por ejemplo, la LDR orientada hacia el interior de la bodega) y, en algunos casos, configurar la lógica para no exigir oscuridad.

Bodega industrial: media. La detección es aplicable, pero se requieren cambios: carcasa con grado de protección IP54 o superior, alimentación de 12/24 V con respaldo, repetidores WiFi o enlace 4G, sensores de mayor alcance (PIR industriales, barreras infrarrojas) y la integración con la red OT bajo los estándares descritos en la sección 6.3.

Factibilidad económica

Tabla 13. *Costo de hardware por nodo (precios referenciales).*

Ítem	Cantidad	Costo unitario ref. (CLP)	Subtotal (CLP)
Arduino UNO R4 WiFi	1	32.000	32.000
Sensor de sonido HW-485	1	2.500	2.500
Sensor de luz HW-486	1	1.500	1.500
Sensor ultrasónico HC-SR04	1	2.500	2.500
Protoboard, cables y conectores	1	4.000	4.000
Fuente USB 5 V / 2 A	1	5.000	5.000
Carcasa plástica	1	5.000	5.000
Celular usado como cámara (opcional, reutilizado)	1	0 – 40.000	0 – 40.000
Total hardware por nodo			**52.500 – 92.500**



Tabla 14. Costos de operación en la nube.

Servicio	Plan	Costo mensual ref.
Servidor Node.js (Render)	Gratuito (se suspende por inactividad) / Starter	USD 0 / ≈ USD 7
Base de datos (MongoDB Atlas)	M0 compartido	USD 0
SMS (Twilio)	Pago por uso	≈ USD 0,05–0,10 por SMS a Chile
Notificaciones push (VAPID)	Estándar web	Sin costo
Conectividad	Internet existente del lugar	Sin costo adicional

Con un costo de hardware cercano a **$52.500 por nodo** y una operación que puede ir de **$0 a ≈ $7.000 mensuales** (más los SMS enviados), la solución resulta claramente más económica que una alarma comercial monitoreada, cuyo servicio en Chile suele costar entre $20.000 y $35.000 mensuales más instalación. En la **vivienda** y el **pequeño comercio** la inversión se recupera en uno o dos meses frente a esa alternativa, por lo que la factibilidad económica es **alta**. En la **bodega industrial** el costo por nodo aumenta (≈ $150.000–$250.000 con carcasa IP, fuente con respaldo, sensores industriales y enlace 4G), además del plan de servidor pagado para evitar la suspensión por inactividad; aun así sigue siendo competitivo frente a sistemas industriales propietarios, por lo que la factibilidad es **media-alta**, condicionada a un piloto exitoso.

Tabla 15. Resumen de factibilidad por entorno.

Entorno	Técnica	Económica	Conclusión
Vivienda	Alta	Alta	Factible con el prototipo actual
Pequeño comercio	Alta / media	Alta	Factible con ajustes de ubicación y lógica
Bodega industrial	Media	Media-alta	Factible con hardware industrial y piloto previo

6.3 Cumplimiento de Estándares OT

Aunque el proyecto es un prototipo académico, su diseño se alineó con buenas prácticas y estándares de Tecnología Operacional (OT), y se identificaron las brechas a cubrir para un despliegue industrial (Tabla 16).

Tabla 16. Alineamiento del proyecto con estándares OT y normativas.

Estándar / norma	Exigencia principal	Alineamiento del proyecto
ISA/IEC 62443 (seguridad de sistemas de automatización y control)	Defensa en profundidad, zonas y conductos, control de acceso, comunicaciones seguras, mínimo privilegio.	Comunicación cifrada TLS (conducto seguro), autenticación con JWT, contraseñas con bcrypt, secretos fuera del código, sin registro público. Brecha: autenticación del dispositivo y segmentación en zonas (VLAN IoT).
Modelo ISA-95 / Purdue	Separación por niveles: campo, control, supervisión y empresa.	Nivel 0: sensores; nivel 1: Arduino (control local de la detección); nivel 2: aplicativo de supervisión; nivel 4: nube. La detección no depende de la nube.
NIST SP 800-82 Rev. 3	Guía de seguridad OT: inventario, monitoreo, gestión de parches y respuesta a incidentes.	Diagnóstico por monitor serie y estado de cada envío; propuesta de OTA y monitoreo de salud de nodos.
IEEE 802.11 / WPA2	Interoperabilidad y seguridad de la red inalámbrica.	Uso de WiFi 802.11 b/g/n certificado con cifrado WPA2.
IEC 60529 (grado IP)	Protección contra polvo y agua según el ambiente.	Prototipo sin grado IP; se recomienda carcasa IP54–IP65 para comercio e industria.
Ley 19.628 y Ley 21.719 (Chile) de protección de datos personales	Tratamiento lícito, consentimiento y resguardo de datos sensibles como la biometría.	Solo se guardan descriptores faciales con consentimiento, con opción de eliminación; acceso restringido por login.
NCh Elec. 4/2003 (instalaciones eléctricas)	Instalaciones de consumo seguras.	Alimentación de 5 V por USB; en industria debe instalarse por electricista autorizado.



Además, se aplicaron buenas prácticas OT durante todo el proceso: el control crítico (detectar la intrusión) se ejecuta localmente en el microcontrolador y no depende de la nube; el firmware nunca se bloquea esperando a la red (operación determinista con ciclo de 50 ms); existe recuperación automática ante fallas de comunicación (backoff exponencial); y los eventos quedan registrados con marca de tiempo y evidencia.

6.4 Impacto de Limitaciones y Restricciones

Tabla 17. *Limitaciones, impacto y estrategias de mitigación.*

Limitación / restricción	Entorno más afectado	Impacto	Estrategia de mitigación
Corte de energía eléctrica	Todos	El sistema se apaga y no detecta ni avisa.	Batería de respaldo (power bank o UPS de 12 V) y aviso de “nodo desconectado”.
Dependencia de WiFi e Internet	Todos, crítico en industria	Sin red no llegan avisos; un intruso podría desconectar el router.	Alarma local (buzzer/sirena), enlace 4G de respaldo y alerta por falta de reportes.
Suspensión por inactividad del plan gratuito del servidor	Comercio e industria	Primera petición tras inactividad tarda varios segundos.	Plan pagado o mantener tráfico constante (el envío cada 1 s ya lo mitiga).
Lógica que exige las tres condiciones a la vez	Comercio (vitrinas iluminadas)	Un intruso con la luz encendida o lejos del sensor no dispara la alarma.	Lógica configurable por zona (2 de 3 condiciones) y sensores PIR adicionales.
Alcance del HC-SR04 (cono ≈ 15°, alerta bajo 50 cm)	Bodega industrial	Cubre solo un punto de paso.	Varios nodos o barreras infrarrojas en accesos.
Ruido ambiente y estructuras metálicas	Bodega industrial	Falsas alarmas por maquinaria; atenuación de la señal WiFi.	Recalibración por turno, antena externa, repetidores y lógica por horario.
Sensores de grado hobby	Comercio e industria	Menor vida útil frente a polvo, humedad y temperatura.	Sensores industriales y carcasa IP54–IP65.
Historial en memoria (20 eventos)	Todos	Se pierde si el servidor se reinicia.	Persistir eventos y fotos en la base de datos o almacenamiento de objetos.
Reconocimiento facial dependiente de iluminación	Vivienda y comercio	Menor precisión de noche.	Cámara con infrarrojo y ajuste del umbral de distancia (0,6).
Cuenta de prueba de SMS	Todos	Solo envía a números verificados.	Cuenta Twilio de pago o canal alternativo (WhatsApp/Telegram).

6.5 Pruebas y Validación

Para validar la implementación en cada entorno se propone un plan de pruebas de campo complementario a las pruebas de laboratorio de las secciones 3.5 y 5.4:

Tabla 18. *Plan de pruebas y validación en campo.*

Método	Entorno	Procedimiento	Criterio de aceptación
Prueba piloto 72 h	Todos	Operar el nodo armado 3 días registrando todos los eventos.	0 caídas no recuperadas; tasa de falsas alarmas ≤ 1 por día.
Intrusión simulada	Todos	Ingreso controlado de una persona al área en distintos horarios (10 repeticiones).	Detección ≥ 9/10; aviso SMS y push en < 10 s.
Latencia extremo a extremo	Todos	Medir con cronómetro desde el evento hasta la notificación en el teléfono.	Dashboard < 2 s; push/SMS < 10 s.
Cobertura WiFi	Comercio e industria	Medir RSSI con WiFi.RSSI() en cada ubicación propuesta.	RSSI mejor que −70 dBm.
Corte de red y energía	Todos	Desconectar el router y la alimentación; restablecer.	Reconexión automática y reanudación de datos en < 2 min.
Ruido de fondo	Bodega industrial	Operar con maquinaria encendida y apagada.	Sin alarmas por ruido de maquinaria sin intrusión.
Seguridad básica	Todos	Intentar acceder a las APIs sin sesión y con token alterado.	Respuestas 401 en todos los casos.



Los resultados de estas pruebas deben documentarse en una planilla con fecha, hora, condición probada, valores registrados en el monitor serie y en el historial del aplicativo, para justificar la puesta en marcha definitiva de cada entorno.

6.6 Recomendaciones

Vivienda

- Instalar el nodo frente al acceso principal, a la altura del pecho, con la LDR orientada hacia el interior.
- Agregar un buzzer o sirena local disuasiva y una batería de respaldo tipo power bank.
- Registrar a los integrantes del hogar en “Personas conocidas” para distinguir rápidamente eventos propios de intrusiones reales.

Pequeño comercio

- Habilitar una lógica configurable (por ejemplo, 2 de 3 condiciones) cuando las vitrinas quedan iluminadas.
- Registrar varios teléfonos para SMS y push (dueño y encargado) y conservar las fotos del historial como evidencia.
- Usar un plan pagado del servidor para evitar la suspensión por inactividad y persistir el historial en la base de datos.

Bodega o recinto industrial

- Migrar a hardware de grado industrial: carcasa IP65, fuente de 24 V con UPS, sensores PIR o barreras infrarrojas y antena externa o enlace 4G.
- Conectar los nodos a una VLAN IoT separada, autenticar cada dispositivo (clave de API o certificado) y adoptar MQTT/OPC UA para integrarse con SCADA o BMS, conforme a IEC 62443.
- Realizar un piloto en una zona antes de escalar, con recalibración por turno y monitoreo de salud de cada nodo.

VII. Conclusiones

El desarrollo del proyecto permitió comprobar que es factible construir un sistema antirrobo funcional y de bajo costo integrando un microcontrolador con conectividad inalámbrica, sensores de uso general y servicios en la nube. El Arduino UNO R4 WiFi resultó una elección adecuada: su lógica de 5 V permitió conectar directamente los tres sensores, y su coprocesador ESP32-S3 resolvió la comunicación WiFi cifrada con TLS sin hardware adicional. La comparación con otras placas mostró que alternativas más potentes o baratas, como el ESP32, habrían requerido adaptar niveles de tensión, mientras que otras no tenían memoria suficiente o estaban sobredimensionadas.

Uno de los principales aportes del trabajo fue la estrategia de detección. Combinar tres condiciones independientes —ruido, oscuridad y cercanía—, exigir tres lecturas consecutivas antes de disparar y aplicar un periodo de rearme redujo notablemente las falsas alarmas y las notificaciones repetidas. A su vez, la calibración automática de umbrales, basada en el reposo y la fluctuación de cada módulo, resolvió un problema real observado en las pruebas: sensores del mismo modelo entregaban lecturas muy distintas. Gracias a ello, el sistema funciona con distintos módulos sin necesidad de reprogramarlo.

En cuanto a la comunicación inalámbrica, el proyecto evidenció que la eficiencia del firmware es tan importante como la elección de la antena. Agrupar cada petición en un único envío, reutilizar la conexión TLS, leer las respuestas sin bloquear y aplicar reintentos con backoff exponencial permitieron mantener un ciclo de detección estable de 50 ms mientras se envían datos cada segundo. La experiencia con la pantalla LCD I2C,



que bloqueaba el sistema hasta 17 segundos, mostró la importancia de validar cada componente y de priorizar la confiabilidad de la función crítica.

La aplicación web progresiva cumplió los requisitos planteados: acceso protegido, estado y lecturas en tiempo real, historial con fotografía, notificaciones push y SMS, cámara en vivo por WebRTC y reconocimiento facial ejecutado en el navegador. Las medidas de seguridad implementadas (HTTPS, bcrypt, JWT en cookie httpOnly, sockets autenticados y secretos en variables de entorno) entregan una base sólida, aunque el análisis identificó mejoras necesarias para un uso productivo, en especial la autenticación del propio dispositivo y la eliminación de las credenciales WiFi del código.

El análisis de entornos concluyó que la solución es plenamente factible en viviendas y pequeños comercios, donde su costo de hardware cercano a $52.500 por nodo y una operación prácticamente gratuita la hacen más económica que una alarma monitoreada tradicional. En recintos industriales la propuesta también es viable, pero requiere hardware de grado industrial, respaldo de energía y de comunicaciones, y un alineamiento más estricto con estándares OT como IEC 62443 y el modelo ISA-95. Las limitaciones identificadas —dependencia de la energía y de Internet, la lógica que exige las tres condiciones y el alcance del sensor ultrasónico— tienen estrategias de mitigación concretas y de bajo costo.

Como proyección, quedan abiertas líneas de profundización que no fue posible abordar en este proceso: la incorporación de una alarma física local, la persistencia completa del historial, el soporte de múltiples nodos y usuarios con roles, la migración a MQTT y la actualización remota del firmware. Estas mejoras permitirían transformar el prototipo en una plataforma de seguridad IoT escalable, capaz de responder a las necesidades de distintos tipos de organizaciones.

VIII. Referencias bibliográficas

Arduino. (2023). *Arduino UNO R4 WiFi: Product reference manual* [Documento técnico]. Recuperado de https://docs.arduino.cc/hardware/uno-r4-wifi

Arduino. (2024). *UNO R4 WiFi network examples (WiFiS3 library)*. Recuperado de https://docs.arduino.cc/tutorials/uno-r4-wifi/wifi-examples

Espressif Systems. (2023). *ESP32-S3 series datasheet* (Versión 1.8). Recuperado de https://www.espressif.com/en/support/documents/technical-documents

ElecFreaks. (s.f.). *Ultrasonic ranging module HC-SR04* [Hoja de datos]. Recuperado de https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

International Electrotechnical Commission. (2018). *IEC 62443-3-3: Industrial communication networks – Network and system security – Part 3-3: System security requirements and security levels*. Ginebra, Suiza: IEC.

International Society of Automation. (2010). *ANSI/ISA-95.00.01-2010: Enterprise-control system integration*. Research Triangle Park, NC: ISA.

Stouffer, K., Pease, M., Tang, C., Zimmerman, T., Pillitteri, V., Lightman, S., Hahn, A., Saravia, S., Sherule, A., y Thompson, M. (2023). *Guide to operational technology (OT) security* (NIST SP 800-82r3). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-82r3

Ley N° 21.719. Regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales. Diario Oficial de la República de Chile, 13 de diciembre de 2024.

Mozilla Developer Network. (2024). *WebRTC API*. Recuperado de https://developer.mozilla.org/es/docs/Web/API/WebRTC_API

Socket.IO. (2024). *Socket.IO documentation v4*. Recuperado de https://socket.io/docs/v4/



Twilio. (2024). *Programmable Messaging API documentation*. Recuperado de https://www.twilio.com/docs/messaging

Mühler, V. (2020). *face-api.js: JavaScript API for face detection and face recognition in the browser* [Software]. Recuperado de https://github.com/justadudewhohacks/face-api.js

Render. (2024). *Render documentation: Free instance types*. Recuperado de https://render.com/docs/free



APRENDER HACIENDO A LO LARGO DE LA VIDA

ACREDITADO
7 AÑOS
MÁXIMA EXCELENCIA

CNA
Comisión Nacional de Acreditación
Chile

INSTITUTO PROFESIONAL INACAP
ACREDITADO • NIVEL DE EXCELENCIA
• Gestión Institucional • Docencia de Pregrado
• Vinculación con el Medio
• Próxima Acreditación Septiembre 2030

www.inacap.cl